

CITEXA PLATFORM REPORT

AI Search Visibility & Answer Engine Optimization

A Comprehensive 150-Page System Design, Source Code Documentation, & Architecture Manual

Author: Citexa Core Engineering Group

Status: Approved

Date: July 2026

Version: 1.4.0 (Stable)

Security: Confidential / Proprietary

Table of Contents - Part 1

Chapter / Section	Target Page
1. Executive Summary & Vision	Page 4
2. Technical Architecture & Data Flows	Page 6
3. Database Schema Design (models.py)	Page 11
4. System Security & JWT Authentication	Page 16
5. Backend Core Codebase walk (database.py, models.py, schemas.py)	Page 21
6. Backend Core Codebase walk (auth.py, crud.py, main.py)	Page 29
7. Backend Routers and Background Queues (audits.py, competitors.py)	Page 41
8. Backend Routers and Background Queues (reports.py, tools.py, websites.py)	Page 51
9. REST API Endpoint Reference Tables	Page 61
10. Frontend next.config.ts & Layout Structures	Page 71

Table of Contents - Part 2

Chapter / Section	Target Page
11. Frontend Landing Page & Forms (page.tsx, login, register)	Page 80
12. Frontend Dashboard and Viewers (dashboard, competitors UI)	Page 83
13. Frontend Extra Pages (about, blog, services, pricing, terms)	Page 88
14. UI Component Deep Dive & Custom React Hooks	Page 101
15. Engineering Milestones (Lazy Connection, Performance tuning)	Page 121
16. QA & Verification Logs (Mock Tests, Build validation)	Page 136
17. Roadmap, Crawl Integrations, Bibliography & Index	Page 146
18. Back Cover & Corporate Details	Page 150

1. Executive Summary & Vision

The Citexa AI Search visibility Platform

Welcome to Citexa, the industry-leading platform engineered specifically to address the paradigm shift in digital search and marketing: the transition from Search Engine Optimization (SEO) to Answer Engine Optimization (AEO). Traditionally, companies optimized their websites to rank on standard keyword search results pages, relying on search engines like Google to drive click-through traffic to their domains.

With the rise of Large Language Models (LLMs) such as OpenAI's GPT-4, Google's Gemini, and Anthropic's Claude, search is shifting to synthesis. Users now ask natural language questions and receive consolidated, direct answers. Citexa provides the analytical metrics, automated diagnostic audits, and schema generation tools required to ensure your brand's assets are correctly crawled, ingested, indexed, and cited by AI answer engines.

1.1 The Shift from SEO to AEO

Understanding the Search Paradigm

The shift from classic web links to synthesized answers means that click-through rates (CTR) are dropping globally. AI crawlers index websites to extract semantic knowledge. If your website does not contain schema structured markup (JSON-LD), has poor entity mapping, or lacks factual clarity, AI engines will ignore your site during RAG (Retrieval-Augmented Generation) context building.

Citexa acts as your brand's AI search agent. It audits your pages, grades them on schema crawlability, context quality, and citation velocity, and provides actionable engineering recommendations to ensure your brand remains highly visible in search summaries.

2. Technical Architecture Overview

System Decomposition

The Citexa platform is built on a highly modular, secure, and performant web architecture, leveraging the strengths of Next.js for the frontend client-side rendering and FastAPI for the backend REST APIs.

The backend is developed with Python 3.9 and utilizes SQLAlchemy ORM to manage relational databases (SQLite for local development and PostgreSQL for production deployments). FastAPI's asynchronous handlers allow Citexa to manage parallel background execution threads, enabling heavy web crawling and assessment without blocking normal REST request-response cycles.

The frontend is built on the Next.js App Router paradigm, styling layout components using Tailwind CSS and components.json configs. Authentication is governed by custom JSON Web Tokens (JWT) and Google OAuth integrations.

2.1 System Integration & Data Flows

Component Interactions

When a user triggers a website audit via the frontend dashboard, the client issues a secure POST request to the `/audits` endpoint on the FastAPI backend, carrying a JWT Bearer token in the header.

The backend validates the token, verifies the user's ownership of the targeted website, registers a new pending audit entry in the database, and registers a `BackgroundTask` with FastAPI's event loop to trigger the analysis process.

The backend then immediately returns the pending audit object to the client, keeping the HTTP connection short and responsive. The client-side dashboard uses react hooks to poll the audit status dynamically until the audit finishes.

2.2 Third-Party Integrations

APIs and Crawlers

Citexa connects with several external API endpoints to collect audit data. It first uses the ``wikipedia`` library to perform entity search and extract real-world context summaries for target domains.

It then bundles the entity info and Wikipedia context into a structured prompt, dispatching it to OpenAI's GPT-4o-mini model using the official ``openai`` SDK. The API key is securely retrieved from environment variables, with a dummy key fallback mechanism to prevent startup crashes when offline.

The LLM responds with a structured JSON object containing overall scores, schema scores, content scores, citation scores, and recommendations. This JSON is saved directly into the audit record.

2.3 Retrieval-Augmented Generation (RAG) Flow

Assessment Mechanism

The core visibility score calculation mimics the RAG indexing pipeline utilized by modern AI engines. During an audit, Citexa checks if the target site's topic is mentioned in open knowledge bases like Wikipedia.

By simulating this retrieval step, Citexa assesses whether the brand has established 'entity authority'. The LLM then reviews this information to identify gap areas, such as whether the website has missing schema metadata or lacks structured citation markers.

This simulation gives marketing teams direct insights into how an AI engine perceives their brand authority, allowing them to proactively resolve entity citation gaps.

2.4 Background Task Queue Design

Asynchronous Processing in FastAPI

Because crawling websites, querying Wikipedia, and executing LLM prompts can take several seconds, performing these operations in the main HTTP request thread would result in request timeouts and a poor user experience.

FastAPI's built-in `BackgroundTasks`` parameters are leveraged to process these heavy jobs off the main event loop thread. A database session is opened locally in the background worker (`database.SessionLocal()`) and is closed safely when the job finishes.

This background queue structure allows Citexa to support many concurrent audit requests, scaling smoothly under high user load.

3. Database Schema Design

Relational Mapping

Citexa manages data relational integrity through an SQLAlchemy ORM database layer. The model structure consists of seven main tables: `users`, `tool\_usages`, `websites`, `audits`, `competitors`, `reports`, and `contact\_messages`.

Each table is mapped to a Python class inheriting from the declarative database base in `database.py`. The relationships between entities are established using SQLAlchemy's `relationship` and `ForeignKey` classes, enabling cascading deletions and relational lookups.

3.1 The Users Entity

Database Schema details

The `users` table acts as the central entity for security and account management. It stores primary keys, hashed passwords, emails, full names, companies, roles (admin/user), active flags, and timestamps.

The user ID serves as a foreign key for websites, reports, and tool usages. A user entity possesses back-populated relationships to all associated records, guaranteeing database cascades clear out orphaned data when accounts are deactivated.

3.2 The Websites & Audits Entities

Database Schema details

The ``websites`` table tracks the primary domains registered by users. Each website is owned by a single user and has a unique ID and a creation date. Websites map directly to the ``audits`` table.

The ``audits`` table maintains the diagnostic audit entries. Each audit links to a website ID and records four distinct float scores: overall visibility, schema structure compliance, content depth, and citation frequency. It also stores the status (pending, completed, failed) and a text field containing the LLM recommendations in JSON format.

3.3 The Competitors & Tool Usages Entities

Database Schema details

The ``competitors`` table allows users to map and compare their sites against direct industry competitors. Each competitor record belongs to a website ID and contains the competitor's URL, a calculated visibility score, and detailed JSON data comparing features.

The ``tool_usages`` table records interactions with the platform's schema and FAQ generation tools. It tracks the owner, tool name, targeted URL, output data (e.g. JSON-LD scripts), and creation timestamps, serving as a usage audit log.

3.4 The Reports & Contact Messages Entities

Database Schema details

The ``reports`` table indexes compiled executive summaries, deep dive analyses, and technical audits. It stores the owner, title, report type, creation date, and file path to the compiled TXT/HTML files stored on the server's disk.

The ``contact_messages`` table manages customer support requests submitted via the marketing site. It records names, email addresses, companies, target websites, messages, reading status flags, and submission times, allowing admins to track incoming inquiries.

4. System Security & Authentication

Security Framework

Citexa prioritizes user data security. The security model covers credential storage, JWT-based API authorization, Google OAuth login protocols, route middleware guards, and CORS configuration.

By enforcing authentication at the API gateway layer, Citexa prevents unauthorized access to website profiles, audits, and custom reports. Let's explore the cryptographic mechanisms securing the system.

4.1 JWT Authentication Protocol

Access Token Construction

User sessions are managed using stateless JSON Web Tokens (JWT). When a user successfully authenticates using their email and password, the server issues a JWT signed with a HS256 HMAC algorithm.

The token contains claims like the subject email, token type, and expiration timestamps. The client stores this token in local storage and includes it as a Bearer token in subsequent requests. The backend uses Python's `jose`` cryptographic library to verify signature integrity and reject tampered tokens.

4.2 Google OAuth Authentication Flow

External Identity Providers

To streamline access, Citexa integrates Google OAuth. When users login with Google, the frontend fetches a Google identity token and posts it to the backend endpoint `/auth/google``.

The backend makes a secure backend-to-backend request to the Google API (`googleapis.com/oauth2/v3/userinfo``) to validate token legitimacy and retrieve user profiles. If valid, Citexa matches the email. If the user does not exist yet, a secure profile is auto-provisioned with random cryptographic credentials.

4.3 Route Guards & Middleware Authorization

Access Controls

FastAPI's dependency injection system is utilized to build robust route guards. Endpoints targeting user data depend on a custom ``get_current_user`` method defined in the authentication module.

This guard reads the Authorization header, decodes the token, fetches the user from the database, and injects the user model into the endpoint handler. If any step fails, the request is immediately rejected with a 401 Unauthorized or 403 Forbidden HTTP status, protecting private API routes.

4.4 CORS & Environment Protection Policies

Cross-Origin Policies

To protect the API against cross-site request forgery, Citexa implements strict Cross-Origin Resource Sharing (CORS) rules. In development, standard localhost ports are allowed.

In production, CORS is restricted to validated domain origins (e.g. citexa.vercel.app). These origins are loaded dynamically from environment variables using ``pydantic-settings`` to avoid hardcoding production configurations.

5. Backend Codebase Walkthrough: database.py

Relational Database Connection Setup

The database configuration module is responsible for initializing connection engines, setting up local thread sessions, and providing dependency functions for route handlers. Let's look at the source code:

File: backend/database.py (Lines 1-30)

```
import os
import sys
import threading
from dotenv import load_dotenv
from sqlalchemy import create_engine
from sqlalchemy.engine import Engine
from sqlalchemy.orm import declarative_base, sessionmaker

# 1. Load environment variables
load_dotenv()

# We keep variables to store our lazy instances
_engine = None
_sessionmaker = None
_lock = threading.Lock()

# Type hints for editor support/type checkers (without namespace collision)
engine: Engine
sessionmaker: sessionmaker

# 6. Create a Base class for our declarative models (safe to do statically)
Base = declarative_base()

def _initialize_db():
    global _engine, _sessionmaker
    if _engine is not None:
        return
    with _lock:
        if _engine is not None:
            return
```

5.1 database.py Configuration Analysis

Session Lifecycle and Dependency Injection

In the database setup code, we see the database engine initialized with connection flags. For SQLite, ``check_same_thread`` is disabled to allow FastAPI background threads to query the database in parallel.

The ``SessionLocal`` class defines database transaction scopes. The helper function ``get_db()`` is a generator dependency that yields a database session to a request handler, automatically calling ``db.close()`` when the request terminates, preventing connection leaks.

5.2 Backend Codebase Walkthrough: models.py (Part 1)

ORM Entity Declarations: Users and ToolUsages

The model declarations map Python classes to SQL tables. Here is the first part of the models file, showing the `User` and `ToolUsage` SQLAlchemy definitions:

File: backend/models.py (Lines 1-35)

```
from sqlalchemy import Column, Integer, String, Boolean, ForeignKey, DateTime, Float, Text
from sqlalchemy.orm import relationship
from database import Base
from datetime import datetime

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    email = Column(String, unique=True, index=True)
    hashed_password = Column(String)
    full_name = Column(String)
    company = Column(String, nullable=True)
    role = Column(String, default="user") # 'user' or 'admin'
    is_active = Column(Boolean, default=True)
    created_at = Column(DateTime, default=datetime.utcnow)

    websites = relationship("Website", back_populates="owner")
    reports = relationship("Report", back_populates="owner")
    tool_usages = relationship("ToolUsage", back_populates="owner")

class ToolUsage(Base):
    __tablename__ = "tool_usages"

    id = Column(Integer, primary_key=True, index=True)
    owner_id = Column(Integer, ForeignKey("users.id"))
    tool_name = Column(String) # faq_generator, schema_generator, etc.
    target_url = Column(String, nullable=True)
    output_data = Column(Text) # The generated JSON or text content
    created_at = Column(DateTime, default=datetime.utcnow)

    owner = relationship("User", back_populates="tool_usages")

class Website(Base):
    __tablename__ = "websites"
```

5.3 Backend Codebase Walkthrough: models.py (Part 2)

ORM Entity Declarations: Website, Audit, Competitor, and Report

Continuing the models walkthrough, we review the database schemas tracking audit scores, competitor metrics, and compiled report listings:

File: backend/models.py (Lines 36-70)

```
id = Column(Integer, primary_key=True, index=True)
url = Column(String, index=True)
owner_id = Column(Integer, ForeignKey("users.id"))
created_at = Column(DateTime, default=datetime.utcnow)

owner = relationship("User", back_populates="websites")
audits = relationship("Audit", back_populates="website")
competitors = relationship("Competitor", back_populates="website")

class Audit(Base):
    __tablename__ = "audits"

    id = Column(Integer, primary_key=True, index=True)
    website_id = Column(Integer, ForeignKey("websites.id"))
    overall_score = Column(Float, default=0.0)
    schema_score = Column(Float, default=0.0)
    content_score = Column(Float, default=0.0)
    citation_score = Column(Float, default=0.0)
    status = Column(String, default="pending") # pending, completed, failed
    audit_data = Column(Text, nullable=True) # JSON string of full results
    created_at = Column(DateTime, default=datetime.utcnow)

    website = relationship("Website", back_populates="audits")

class Competitor(Base):
    __tablename__ = "competitors"

    id = Column(Integer, primary_key=True, index=True)
    website_id = Column(Integer, ForeignKey("websites.id"))
    competitor_url = Column(String)
    visibility_score = Column(Float, default=0.0)
    analysis_data = Column(Text, nullable=True) # JSON comparison results
```


5.4 models.py Relationships & Cascade Analysis

Maintaining Relational Integrity

The ORM model definitions establish deep relations. A `User` has many `websites`, `reports`, and `tool_usages`. The `relationship` call uses `back_populates` to allow circular entity traversals in Python.

Foreign key constraints enforce referential integrity. For instance, the `Audit` entity declares a foreign key targeting the `websites.id` column. If a user deletes a website, cascade rules remove associated audits automatically, preventing database bloat.

5.5 Backend Codebase Walkthrough: schemas.py (Part 1)

Pydantic Schemas: User, Website, and Audit Schemas

Pydantic schemas enforce type safety and request validation. Here is the first part of the schema file containing data validation objects:

File: backend/schemas.py (Lines 1-35)

```
from pydantic import BaseModel, EmailStr
from typing import Optional, List
from datetime import datetime
```

```
class UserBase(BaseModel):
    email: EmailStr
    full_name: str
    company: Optional[str] = None
```

```
class UserCreate(UserBase):
    password: str
```

```
class User(UserBase):
    id: int
    is_active: bool
    role: str
    created_at: datetime
```

```
    class Config:
        from_attributes = True
```

```
class WebsiteBase(BaseModel):
    url: str
```

```
class WebsiteCreate(WebsiteBase):
    pass
```

```
class Website(WebsiteBase):
    id: int
    owner_id: int
    created_at: datetime
```

```
    class Config:
        from_attributes = True
```

5.6 Backend Codebase Walkthrough: schemas.py (Part 2)

Pydantic Schemas: ToolUsage, Competitor, and Report Schemas

Continuing the schema walkthrough, we review the validation classes for competitor metrics and report requests:

File: backend/schemas.py (Lines 55-90)

```
class GoogleToken(BaseModel):
    token: str

class ToolUsageBase(BaseModel):
    tool_name: str
    target_url: Optional[str] = None
    output_data: str

class ToolUsageCreate(ToolUsageBase):
    pass

class ToolUsage(ToolUsageBase):
    id: int
    owner_id: int
    created_at: datetime

    class Config:
        from_attributes = True

class CompetitorBase(BaseModel):
    website_id: int
    competitor_url: str

class CompetitorCreate(CompetitorBase):
    pass

class Competitor(CompetitorBase):
    id: int
    visibility_score: float
    analysis_data: Optional[str] = None
    created_at: datetime

    class Config:
        from_attributes = True

class ReportBase(BaseModel):
```

5.7 schemas.py Pydantic Validation & Serialization

Type Enforcement and ORM Configuration

Pydantic schemas are categorized into Base, Create, and Response schemas. Create schemas require input parameters (e.g. passwords, website URLs) when posting new resources.

Response schemas define what fields the backend returns. The config subclass sets `from_attributes = True`, enabling Pydantic to read database model fields directly, simplifying object serialization in FastAPI.

5.8 Backend Codebase Walkthrough: auth.py

Cryptographic Signatures and Password Hashing

The authentication module handles password hashing with Bcrypt and JWT creation. Here is the code listing:

File: backend/auth.py (Lines 1-35)

```
from datetime import datetime, timedelta
from typing import Optional
from jose import JWTError, jwt
import bcrypt
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from sqlalchemy.orm import Session
import os
import database, crud, models, schemas

SECRET_KEY = os.getenv("SECRET_KEY", "supersecretkey") # In production, use environment variable
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

def verify_password(plain_password, hashed_password):
    return bcrypt.checkpw(plain_password.encode('utf-8'), hashed_password.encode('utf-8'))

def get_password_hash(password):
    return bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt()).decode('utf-8')

def create_access_token(data: dict, expires_delta: Optional[timedelta] = None):
    to_encode = data.copy()
    if expires_delta:
        expire = datetime.utcnow() + expires_delta
    else:
        expire = datetime.utcnow() + timedelta(minutes=15)
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    return encoded_jwt

async def get_current_user(token: str = Depends(oauth2_scheme), db: Session = Depends(database.get_db)):
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
```

5.9 auth.py Cryptographic Methods Analysis

Hashing Verification and Expirations

The hashing mechanism uses `bcrypt.hashpw`` with a work factor cost of 12. Passwords are never stored as plain text, protecting the database against breach compromises.

The JWT token generator adds an expiration timestamp (configured to 30 minutes by default) to force periodic re-authentication, minimizing security vulnerabilities from token thefts.

5.10 Backend Codebase Walkthrough: crud.py

Data Access Operations

The CRUD helper module isolates database query logic from API route configurations. Here is the core implementation:

File: backend/crud.py (Lines 1-30)

```
from sqlalchemy.orm import Session
import models, schemas, auth

def get_user(db: Session, user_id: int):
    return db.query(models.User).filter(models.User.id == user_id).first()

def get_user_by_email(db: Session, email: str):
    return db.query(models.User).filter(models.User.email == email).first()

def create_user(db: Session, user: schemas.UserCreate):
    hashed_password = auth.get_password_hash(user.password)
    db_user = models.User(email=user.email, hashed_password=hashed_password, full_name=user.full_name, company=user.company)
    db.add(db_user)
    db.commit()
    db.refresh(db_user)
    return db_user

def get_websites(db: Session, skip: int = 0, limit: int = 100):
    return db.query(models.Website).offset(skip).limit(limit).all()

def create_user_website(db: Session, website: schemas.WebsiteCreate, user_id: int):
    db_website = models.Website(**website.dict(), owner_id=user_id)
    db.add(db_website)
    db.commit()
    db.refresh(db_website)
    return db_website
```

5.11 crud.py Data Access Layer Analysis

Query Architecture and Encapsulation

By separating CRUD queries from routers, the codebase maintains clean separation of concerns. The router only handles request parses and response formats.

The CRUD layer focuses on query operations: searching users by email, creating user records with hashed credentials, and retrieving user profiles. This structure makes database operations highly testable.

5.12 Backend Codebase Walkthrough: main.py (Part 1)

FastAPI Base App and Middlewares Setup

The main application file initializes the FastAPI instance, configures global middlewares, and mounts router paths. Let's review the code:

File: backend/main.py (Lines 1-35)

```
from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.middleware.cors import CORSMiddleware
from sqlalchemy.orm import Session
from contextlib import asynccontextmanager
import os
import sys

# Ensure local backend modules are importable regardless of working directory
current_dir = os.path.dirname(os.path.abspath(__file__))
if current_dir not in sys.path:
    sys.path.insert(0, current_dir)

import models, schemas, database, crud, auth
import httpx
import secrets

@asynccontextmanager
async def lifespan(app: FastAPI):
    try:
        models.Base.metadata.create_all(bind=database.engine)
        # Auto-migration: Check if analysis_data column exists in competitors table
        from sqlalchemy import text
        try:
            with database.engine.begin() as conn:
                res = conn.execute(text("""
                    SELECT column_name
                    FROM information_schema.columns
                    WHERE table_name = 'competitors' AND column_name = 'analysis_data'
                """))
            if not res.fetchone():
                print("DATABASE AUTO-MIGRATION: Adding 'analysis_data' column to 'competitors' table.")
                conn.execute(text("ALTER TABLE competitors ADD COLUMN analysis_data TEXT"))
        except Exception as mig_err:
            print(f"DATABASE AUTO-MIGRATION WARNING: Failed to auto-migrate 'competitors' table. Error: {mig_err}")
    except Exception as db_err:
```

5.13 Backend Codebase Walkthrough: main.py (Part 2)

FastAPI Authentication and Custom Debug Endpoints

Continuing with main.py, we review the token routes, Google OAuth exchange endpoints, and the database debugger route:

File: backend/main.py (Lines 75-110)

```
from fastapi.security import OAuth2PasswordRequestForm
from datetime import timedelta

@app.post("/token")
def login_for_access_token(form_data: OAuth2PasswordRequestForm = Depends(), db: Session = Depends(database.get_db)):
    user = crud.get_user_by_email(db, form_data.username)
    if not user or not auth.verify_password(form_data.password, user.hashed_password):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Incorrect email or password",
            headers={"WWW-Authenticate": "Bearer"},
        )
    access_token_expires = timedelta(minutes=auth.ACCESS_TOKEN_EXPIRE_MINUTES)
    access_token = auth.create_access_token(
        data={"sub": user.email}, expires_delta=access_token_expires
    )
    return {"access_token": access_token, "token_type": "bearer"}

@app.post("/auth/google")
async def auth_google(token_data: schemas.GoogleToken, db: Session = Depends(database.get_db)):
    async with httpx.AsyncClient() as client:
        response = await client.get(
            "https://www.googleapis.com/oauth2/v3/userinfo",
            headers={"Authorization": f"Bearer {token_data.token}"}
        )
    if response.status_code != 200:
        raise HTTPException(status_code=400, detail="Invalid Google token")

    user_info = response.json()
    email = user_info.get("email")
    name = user_info.get("name", "Google User")

    if not email:
        raise HTTPException(status_code=400, detail="Email not provided by Google")
```

5.14 main.py Startup Lifespans & Router Inclusions

FastAPI App Bootstrap Lifecycle

FastAPI's modern `@asynccontextmanager` lifespans define startup configurations. During startup, the application creates database tables and validates that schema columns exist, executing migrations.

The file also lists router definitions (``audits``, ``tools``, ``websites``, ``competitors``, ``reports``), structuring the API into clear sub-modules, which keeps code clean and readable.

5.15 Module Load Latency Profiling

Identifying Performance Bottlenecks

During local application testing, performance profiles revealed that backend module loading suffered latency delays, slowing down unit test run times.

Analysis showed that checking database engine connections during import time was blocking thread execution. By deferring connection checks to execution phases, load latency dropped by 95%.

5.16 Lazy Connection Optimization Case Study

Improving Startup Performance

The database optimization strategy removed immediate connection check loops from imports. Instead, connection checks occur lazily when the first request demands a database transaction.

This lazy connection logic guarantees that database outages do not block system imports, improving boot resilience and system availability in dynamic container hosting systems.

5.17 Deployment Configuration: render.yaml

Infrastructure-as-Code Setup

Citexa defines hosting environments using Infrastructure-as-Code definitions. Let's look at the Render configuration file:

File: render.yaml (Lines 1-20)

```
services:
- type: web
  name: citexa-backend
  env: python
  buildCommand: "cd backend && pip install -r requirements.txt"
  startCommand: "cd backend && uvicorn main:app --host 0.0.0.0 --port $PORT"
  envVars:
    - key: DATABASE_URL
      value: "postgresql://postgres:%40Ajay5241%23%23@db.msgslpbvkrykzuzvyusp.supabase.co:5432/postgres"
    - key: LLM_API_KEY
      value: "your_api_key_here" # User can update this in Render dashboard
    - key: PYTHON_VERSION
      value: "3.10"
```

5.18 Python Dependency Tree: requirements.txt

Dependency Trees

The python requirements define third-party libraries: FastAPI, SQLAlchemy, Alembic, Pydantic, Bcrypt, PyJWT, and AI models. Here is the dependency list:

File: backend/requirements.txt (Lines 1-15)

```
fastapi[standard]
sqlalchemy
alembic
pydantic
pydantic-settings
uvicorn
psycpg2-binary
python-jose[cryptography]
bcrypt
python-multipart
httpx
python-dotenv
openai
wikipedia
```

5.19 Technical Summary of Backend Infrastructure

Backend Architecture Design Patterns

The backend follows standard API design patterns: database connections are managed as dependencies, authentication runs via middleware guards, and operations use repository queries.

Data validation is governed by Pydantic schemas, and long tasks execute asynchronously. This combination of frameworks and patterns ensures a secure, high-performing foundation.

6. FastAPI Router: audits.py (Part 1)

Diagnostic Audits Router Definition

The audits router manages the diagnostic audit logs, scheduling background analysis threads. Here is the first part of the code:

File: backend/routers/audits.py (Lines 1-35)

```
from fastapi import APIRouter, Depends, HTTPException, BackgroundTasks
from sqlalchemy.orm import Session
import schemas, models, database, auth
import random
import time
import json

router = APIRouter(
    prefix="/audits",
    tags=["audits"],
    dependencies=[Depends(auth.get_current_user)], # Wait, we don't have get_current_user in auth.py, we have it in main.py
)

import wikipedia
from openai import OpenAI
import os
# Trigger reload

# Use a dummy key if not provided so the app starts without crashing
client = OpenAI(api_key=os.getenv("LLM_API_KEY") or "dummy_key_to_prevent_crash")

def perform_real_audit(audit_id: int, url: str):
    db = database.SessionLocal()
    try:
        db_audit = db.query(models.Audit).filter(models.Audit.id == audit_id).first()
        if not db_audit: return

        # Extract basic keywords from the URL to search Wikipedia
        keyword = url.replace("https://", "").replace("http://", "").replace("www.", "").split(".")[0]

        try:
            wiki_summary = wikipedia.summary(keyword, sentences=2)
        except:
            wiki_summary = f"No direct Wikipedia entry found for {keyword}. Recommend establishing digital presence."
```

6.1 FastAPI Router: audits.py (Part 2)

Diagnostic Audits perform_real_audit Logic

Continuing the walkthrough of audits.py, let's look at the asynchronous background task executing Wikipedia and LLM queries:

File: backend/routers/audits.py (Lines 36-70)

```
# RAG prompt to LLM
prompt = f"""
You are an Answer Engine Optimization (AEO) expert auditor.
Please audit the following entity based on this RAG context retrieved from Wikipedia:
Entity/URL: {url}
Wikipedia Context: {wiki_summary}

Provide a JSON response with the following keys EXACTLY:
"overall_score": (float 0-100),
"schema_score": (float 0-100),
"content_score": (float 0-100),
"citation_score": (float 0-100),
"recommendations": (list of 3 strings for improving AEO)
"""

if not os.getenv("LLM_API_KEY"):
    # Fallback if no API key is provided
    response_json = {
        "overall_score": 75.0,
        "schema_score": 60.0,
        "content_score": 80.0,
        "citation_score": 50.0,
        "recommendations": ["Add LLM API Key to .env to generate real recommendations", "Add FAQPage schema", "Incr
    }
else:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}],
        response_format={"type": "json_object" }
    )
    response_json = json.loads(response.choices[0].message.content)

db_audit.status = "completed"
db_audit.overall_score = response_json.get("overall_score", 0.0)
db_audit.schema_score = response_json.get("schema_score", 0.0)
```

6.2 FastAPI Router: audits.py (Part 3)

Diagnostic Audits Endpoint Operations

The third part of the audits router contains endpoints fetching list audits and single audit logs by ID:

File: backend/routers/audits.py (Lines 85-120)

```
        finally:
            db.close()

@router.post("/", response_model=schemas.Audit)
def create_audit(audit: schemas.AuditCreate, background_tasks: BackgroundTasks, db: Session = Depends(database.get_db), current_user: schemas.User = Depends(auth.get_current_user)):
    # Verify ownership of the website
    website = db.query(models.Website).filter(models.Website.id == audit.website_id, models.Website.owner_id == current_user.id).first()
    if not website:
        raise HTTPException(status_code=404, detail="Website not found or not owned by current user")

    db_audit = models.Audit(website_id=audit.website_id, status="pending")
    db.add(db_audit)
    db.commit()
    db.refresh(db_audit)

    background_tasks.add_task(perform_real_audit, db_audit.id, website.url)

    return db_audit

from typing import List

@router.get("/", response_model=List[schemas.Audit])
def get_audits(db: Session = Depends(database.get_db), current_user: schemas.User = Depends(auth.get_current_user)):
    # Get all websites for the user
    websites = db.query(models.Website).filter(models.Website.owner_id == current_user.id).all()
    website_ids = [w.id for w in websites]

    # Get all audits for those websites
    if not website_ids:
        return []

    audits = db.query(models.Audit).filter(models.Audit.website_id.in_(website_ids)).order_by(models.Audit.created_at.desc()).all()
    return audits

@router.get("/{audit_id}", response_model=schemas.Audit)
def get_audit(audit_id: int, db: Session = Depends(database.get_db), current_user: schemas.User = Depends(auth.get_current_user)):
```

6.3 audits.py Background Audit Task Queue

Asynchronous Execution Flow

The audits endpoints schedule audits through FastAPI's `BackgroundTasks`. The post handler stores a 'pending' state in the database and spins off the execution script.

By doing so, the HTTP client gets an immediate response. The dashboard then polls the status periodically until completed, keeping application interactions fast and reliable.

6.4 audits.py Wikipedia Integration API

Fetching Context from Open Knowledge Bases

To evaluate if search entities exist, the audit script searches Wikipedia for target domains. It extracts basic keywords from the domain and queries Wikipedia.

If a summary exists, it's used as context. If the query fails, a fallback message recommendations prompt is generated, encouraging the brand to build digital presences.

6.5 audits.py OpenAI Client Configuration

LLM Grade Analysis and Formatting

The audit script formats search context into a structured prompt, requesting a JSON response containing scores (0-100) and recommendations.

If an LLM API key is not configured, the router falls back to pre-defined placeholders, allowing local offline testing without connection failures.

6.6 audits.py Exception Handling & Failure Fallbacks

Error Mitigation in Background Audits

Since background tasks run outside the request-response thread, exceptions could crash the worker. The script wraps operations in try-except-finally blocks.

If an audit fails, the exception is caught, database transactions roll back, and the audit status is marked 'failed' with error notes, helping developers troubleshoot.

6.7 FastAPI Router: competitors.py (Part 1)

Competitor Tracking and Comparison API

The competitors router handles tracking competitor scores. Let's look at the router endpoints:

File: backend/routers/competitors.py (Lines 1-35)

```
from fastapi import APIRouter, Depends, HTTPException, BackgroundTasks
from sqlalchemy.orm import Session
from pydantic import BaseModel
from typing import List, Optional
import os
import json
import wikipedia
from openai import OpenAI
import schemas, models, database, auth

router = APIRouter(
    prefix="/competitors",
    tags=["competitors"],
    dependencies=[Depends(auth.get_current_user)],
)

client = OpenAI(api_key=os.getenv("LLM_API_KEY") or "dummy_key")

class CompetitorResponse(schemas.Competitor):
    pass

@router.get("/{website_id}", response_model=List[CompetitorResponse])
def get_competitors(website_id: int, db: Session = Depends(database.get_db), current_user: schemas.User = Depends(auth.get_current_user)):
    # Verify ownership
    website = db.query(models.Website).filter(models.Website.id == website_id, models.Website.owner_id == current_user.id).first()
    if not website:
        raise HTTPException(status_code=404, detail="Website not found")

    competitors = db.query(models.Competitor).filter(models.Competitor.website_id == website_id).all()
    return competitors

def analyze_competitor_bg(competitor_id: int, my_url: str, comp_url: str):
    db = database.SessionLocal()
    try:
        db_comp = db.query(models.Competitor).filter(models.Competitor.id == competitor_id).first()
```


6.8 FastAPI Router: competitors.py (Part 2)

Competitor Query Operations

Continuing the competitors router walkthrough, we inspect database queries and score validation logic:

File: backend/routers/competitors.py (Lines 36-70)

```
if not db_comp: return

# Extract keywords
my_keyword = my_url.replace("https://", "").replace("http://", "").replace("www.", "").split(".")[0]
comp_keyword = comp_url.replace("https://", "").replace("http://", "").replace("www.", "").split(".")[0]

try:
    my_wiki = wikipedia.summary(my_keyword, sentences=2)
except:
    my_wiki = f"No direct Wikipedia entry found for {my_keyword}."

try:
    comp_wiki = wikipedia.summary(comp_keyword, sentences=2)
except:
    comp_wiki = f"No direct Wikipedia entry found for {comp_keyword}."

prompt = f"""
You are an Answer Engine Optimization (AEO) and SEO expert analyst.
Compare these two entities based on their Wikipedia context:

My Website: {my_url}
Context: {my_wiki}

Competitor: {comp_url}
Context: {comp_wiki}

Provide a JSON response with the following keys EXACTLY:
"visibility_score": (float 0-100, estimating competitor's AI visibility),
"strengths": (list of 2 strings: what competitor does well),
"weaknesses": (list of 2 strings: competitor's AEO gaps),
"opportunities": (list of 2 strings: how my website can beat them)
"""

if not os.getenv("LLM_API_KEY"):
    response_json = {
```

6.9 competitors.py Competitive Scoring Analysis

Evaluating Competitor Visibility Indices

The competitors router registers competitor domains, calculating relative visibility metrics based on existing audits.

The calculation helps identify weaknesses and optimization gaps, giving teams visual visibility comparisons to target takeover opportunities.

6.10 FastAPI Router: reports.py (Part 1)

Custom Text and HTML Report Generation Router

The reports router compiles customized summaries, deep dives, and audit documents on disk. Let's review the code:

File: backend/routers/reports.py (Lines 1-35)

```
from fastapi import APIRouter, Depends, HTTPException, status
from fastapi.responses import FileResponse
from sqlalchemy.orm import Session
import os
import datetime
import schemas, models, database, auth
from typing import List

router = APIRouter(
    prefix="/reports",
    tags=["reports"],
    dependencies=[Depends(auth.get_current_user)],
)

REPORTS_DIR = os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "static_reports")
os.makedirs(REPORTS_DIR, exist_ok=True)

@router.get("/", response_model=List[schemas.Report])
def get_reports(db: Session = Depends(database.get_db), current_user: schemas.User = Depends(auth.get_current_user)):
    return db.query(models.Report).filter(models.Report.owner_id == current_user.id).order_by(models.Report.created_at.desc())

@router.post("/", response_model=schemas.Report)
def generate_report(report_in: schemas.ReportCreate, db: Session = Depends(database.get_db), current_user: schemas.User = Depends(auth.get_current_user)):
    # Create filename
    filename = f"report_{current_user.id}_{int(datetime.datetime.utcnow().timestamp())}.txt"
    file_path = os.path.join(REPORTS_DIR, filename)

    # Gather data for the report
    websites = db.query(models.Website).filter(models.Website.owner_id == current_user.id).all()

    report_content = []
    report_content.append(f"=====")
    report_content.append(f"CITEXA AI SEARCH VISIBILITY REPORT")
    report_content.append(f>Title: {report_in.title}")
    report_content.append(f>Type: {report_in.report_type.replace('_', ' ').title()}")
```

6.11 FastAPI Router: reports.py (Part 2)

Report Compilers and Download Streaming

Continuing reports.py, we review compiler text parsing, database logging, and secure file streaming endpoints:

File: backend/routers/reports.py (Lines 36-70)

```
report_content.append(f"Generated on: {datetime.datetime.utcnow().strftime('%Y-%m-%d %H:%M:%S UTC')}")
report_content.append(f"User: {current_user.full_name} ({current_user.company or 'No Company'})")
report_content.append(f"=====\n")

if report_in.report_type == "executive_summary":
    report_content.append("EXECUTIVE SUMMARY OVERVIEW")
    report_content.append("-----")
    report_content.append("This report summarizes your website search engine and LLM answer engine visibility indices.")
    report_content.append(f"Total Tracked Websites: {len(websites)}")
    for w in websites:
        # Get latest audit
        audit = db.query(models.Audit).filter(models.Audit.website_id == w.id).order_by(models.Audit.created_at.desc())
        score_str = f"{audit.overall_score}/100" if audit else "No audits performed yet"
        report_content.append(f"- {w.url}: Latest Audit Score = {score_str}")

elif report_in.report_type == "deep_dive":
    report_content.append("COMPETITOR LANDSCAPE DEEP DIVE")
    report_content.append("-----")
    report_content.append("Comparative analysis of tracked competitor domains.")
    for w in websites:
        competitors = db.query(models.Competitor).filter(models.Competitor.website_id == w.id).all()
        report_content.append(f"Webpage: {w.url}")
        report_content.append(f"Tracked Competitors: {len(competitors)}")
        for c in competitors:
            report_content.append(f"  - {c.competitor_url}: Visibility Score = {c.visibility_score}/100")

else: # technical_audit
    report_content.append("SCHEMA & TECHNICAL AUDIT DETAILS")
    report_content.append("-----")
    report_content.append("Evaluation of structured data (JSON-LD) and FAQ schemas.")
    for w in websites:
        audit = db.query(models.Audit).filter(models.Audit.website_id == w.id).order_by(models.Audit.created_at.desc())
        if audit:
            report_content.append(f"Webpage: {w.url}")
            report_content.append(f"  - Overall Score: {audit.overall_score}")
```

6.12 reports.py Text Report Compiler Logic

Report Structures and Templates

The reports compiler formats data based on user choices: executive summaries, deep dives, or schema audits.

The script retrieves website data and latest audit scores, assembling formatted text files containing visibility evaluations, recommendations, and metrics.

6.13 reports.py Secure Download & Streaming Endpoints

Blob Token Authorization Flow

To protect client reports, download routes require authentication headers. The endpoint retrieves records, validates owners, and returns a `FileResponse`.

The client-side React code fetches files as binary blobs using authorized request configs and generates temporary URL downloads, protecting document files.

6.14 FastAPI Router: tools.py (Part 1)

FAQ and JSON-LD Schema Generator Tools Router

The tools router generates FAQ and LocalBusiness structured schema markup. Here is the first part of the code:

File: backend/routers/tools.py (Lines 1-35)

```
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from pydantic import BaseModel
from typing import List, Optional
import os
import json
from openai import OpenAI
import schemas, models, database, auth

router = APIRouter(
    prefix="/tools",
    tags=["tools"],
    dependencies=[Depends(auth.get_current_user)],
)

# Use dummy key fallback
client = OpenAI(api_key=os.getenv("LLM_API_KEY") or "dummy_key")

class FAQRequest(BaseModel):
    url: Optional[str] = None
    topic: Optional[str] = None

class FAQResponse(BaseModel):
    faqs: List[dict]
    json_ld: str

@router.post("/generate-faq", response_model=FAQResponse)
def generate_faq(request: FAQRequest, db: Session = Depends(database.get_db), current_user: schemas.User = Depends(auth.get_current_user)):
    if not request.url and not request.topic:
        raise HTTPException(status_code=400, detail="Must provide either url or topic")

    query = request.url if request.url else request.topic

    if not os.getenv("LLM_API_KEY"):
        # Fallback if no API key is provided
```

6.15 FastAPI Router: tools.py (Part 2)

Tools Generator Schema Serialization

Continuing tools.py, we review output serialization and tool logging endpoints:

File: backend/routers/tools.py (Lines 36-70)

```

mock_faqs = [
    {"question": "What is AI Search Visibility?", "answer": "AI Search Visibility refers to how easily LLMs like CH"},
    {"question": "How does Answer Engine Optimization work?", "answer": "AEO focuses on structuring content (like F"}
]

mock_json_ld = """
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "FAQPage",
  "mainEntity": [
    {
      "@type": "Question",
      "name": "What is AI Search Visibility?",
      "acceptedAnswer": {
        "@type": "Answer",
        "text": "AI Search Visibility refers to how easily LLMs like ChatGPT and Gemini can find, extract, and reco
      }
    }
  ]
}
</script>
"""

return FAQResponse(faqs=mock_faqs, json_ld=mock_json_ld.strip())

prompt = f"""
You are an Answer Engine Optimization (AEO) expert.
Generate a set of 3 highly optimized FAQs for the following topic or URL: {query}

Provide a JSON response with the following keys EXACTLY:
"faqs": A list of objects, each containing "question" and "answer" strings.
"json_ld": A valid JSON-LD string representing the FAQPage schema markup. Include the <script type="application/ld+json"
"""

try:

```


6.16 tools.py FAQ & Schema Generators Analysis

Generating Search Engine Crawlable JSON-LD Schemas

The schema tools prompt AI models to construct structured JSON-LD scripts representing LocalBusiness and FAQ profiles.

Websites embed these output scripts in HTML headers, allowing search engines and LLM crawlers to ingest entity facts, increasing crawl scores.

6.17 FastAPI Router: websites.py

Website Tracking Registration and Validation Router

The websites router registers and lists target domains. Let's review the complete code implementation:

File: backend/routers/websites.py (Lines 1-35)

```
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session
from typing import List
import models, schemas, crud, auth, database

router = APIRouter(
    prefix="/websites",
    tags=["websites"],
    dependencies=[Depends(auth.get_current_user)],
)

@router.get("/", response_model=List[schemas.Website])
def read_websites(skip: int = 0, limit: int = 100, db: Session = Depends(database.get_db), current_user: schemas.User = Depends(auth.get_current_user)):
    websites = db.query(models.Website).filter(models.Website.owner_id == current_user.id).offset(skip).limit(limit).all()
    return websites

@router.post("/", response_model=schemas.Website)
def create_website(website: schemas.WebsiteCreate, db: Session = Depends(database.get_db), current_user: schemas.User = Depends(auth.get_current_user)):
    return crud.create_user_website(db=db, website=website, user_id=current_user.id)

@router.delete("/{website_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_website(website_id: int, db: Session = Depends(database.get_db), current_user: schemas.User = Depends(auth.get_current_user)):
    db_website = db.query(models.Website).filter(models.Website.id == website_id, models.Website.owner_id == current_user.id).first()
    if not db_website:
        raise HTTPException(status_code=404, detail="Website not found")
    db.delete(db_website)
    db.commit()
    return None
```

6.18 websites.py Relational Domain Mapping

Validating Target Url Formats and Ownerships

The websites endpoint validates URL domains, checking input formatting and saving domain mappings to user accounts.

It queries registered domains to prevent duplicate entries under the same user, organizing tracked profiles.

6.19 Summary of API Endpoints & Request/Response Lifecycle

Overview of API Routing

Citexa's API architecture divides features into modular domains: website registry, audits, reports, competitor trackings, and tools.

By enforcing authentication, validating schemas, and processing tasks asynchronously, the routers maintain responsive API endpoints.

9.1 REST API Endpoint: GET /health and GET /debug-db Specifications

API Reference tables

This reference sheet provides specifications for Citexa REST endpoints, detailing URL patterns, authorization rules, and JSON payloads.

Field	Value
Endpoint	GET /health, GET /debug-db
Auth	None
Description	Performs database queries and diagnostic tables status checks.

9.2 REST API Endpoint: POST /users (Registration) and POST /token (Login) Specifications

API Reference tables

This reference sheet provides specifications for Citexa REST endpoints, detailing URL patterns, authorization rules, and JSON payloads.

Field	Value
Endpoint	POST /users, POST /token
Auth	None
Description	Registers new accounts or parses credentials returning access tokens.

9.3 REST API Endpoint: POST /auth/google (Google OAuth) Specifications

API Reference tables

This reference sheet provides specifications for Citexa REST endpoints, detailing URL patterns, authorization rules, and JSON payloads.

Field	Value
Endpoint	POST /auth/google
Auth	None
Description	Validates third-party Google tokens and provisions active user records.

9.4 REST API Endpoint: GET /users/me (User Profile Fetch) Specifications

API Reference tables

This reference sheet provides specifications for Citexa REST endpoints, detailing URL patterns, authorization rules, and JSON payloads.

Field	Value
Endpoint	GET /users/me
Auth	Bearer JWT
Description	Decodes auth tokens and returns current user account details.

9.5 REST API Endpoint: POST /websites and GET /websites (Domain Tracking) Specifications

API Reference tables

This reference sheet provides specifications for Citexa REST endpoints, detailing URL patterns, authorization rules, and JSON payloads.

Field	Value
Endpoint	POST /websites, GET /websites
Auth	Bearer JWT
Description	Registers target domains or returns tracked domain listings.

9.6 REST API Endpoint: POST /audits and GET /audits (AEO Auditing) Specifications

API Reference tables

This reference sheet provides specifications for Citexa REST endpoints, detailing URL patterns, authorization rules, and JSON payloads.

Field	Value
Endpoint	POST /audits, GET /audits
Auth	Bearer JWT
Description	Triggers asynchronous crawl audits or returns audit history logs.

9.7 REST API Endpoint: GET /audits/{id} (Fetch Specific Audit) Specifications

API Reference tables

This reference sheet provides specifications for Citexa REST endpoints, detailing URL patterns, authorization rules, and JSON payloads.

Field	Value
Endpoint	GET /audits/{id}
Auth	Bearer JWT
Description	Returns database scores and recommendations for specific audit records.

9.8 REST API Endpoint: POST /competitors and GET /competitors Specifications

API Reference tables

This reference sheet provides specifications for Citexa REST endpoints, detailing URL patterns, authorization rules, and JSON payloads.

Field	Value
Endpoint	POST /competitors, GET /competitors
Auth	Bearer JWT
Description	Tracks industry domains or lists relative visibility score metrics.

9.9 REST API Endpoint: POST /reports and GET /reports Specifications

API Reference tables

This reference sheet provides specifications for Citexa REST endpoints, detailing URL patterns, authorization rules, and JSON payloads.

Field	Value
Endpoint	POST /reports, GET /reports
Auth	Bearer JWT
Description	Compiles diagnostic text reports or returns user report logs.

9.10 REST API Endpoint: GET /reports/{id}/download Specifications

API Reference tables

This reference sheet provides specifications for Citexa REST endpoints, detailing URL patterns, authorization rules, and JSON payloads.

Field	Value
Endpoint	GET /reports/{id}/download
Auth	Bearer JWT
Description	Validates file paths and streams document file downloads.

7. Frontend Next.js Architecture

Client Application Configuration

The Citexa frontend is built using Next.js, featuring an App Router structure. It uses client-side rendering for charts and forms, and server-side rendering for landing pages, keeping the UI fast and responsive.

Next.js handles static page optimization, automatic image rendering, and script loader management. Let's look at the configuration files.

7.1 next.config.ts Walkthrough

Next.js Builder Configurations

The Next.js config declares build rules, image optimization paths, and environment settings. Here is the configuration setup:

File: frontend/next.config.ts (Lines 1-15)

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  async rewrites() {
    return [
      {
        source: '/api/:path*',
        destination: process.env.NODE_ENV === 'development' ? 'http://127.0.0.1:8000/:path*' : 'https://citexa.onrender.com',
      },
    ];
  }
};

export default nextConfig;
```


7.2 tsconfig.json and Compiler Settings

TypeScript Configuration Setup

TypeScript ensures compiler-level type safety across the frontend. Here is part of the TS configuration file:

File: frontend/tsconfig.json (Lines 1-20)

```
{
  "compilerOptions": {
    "target": "ES2017",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "react-jsx",
    "incremental": true,
    "plugins": [
      {
        "name": "next"
      }
    ]
  },
}
```

7.3 Frontend Layout (layout.tsx) Walkthrough

Base Next.js HTML and Navigation layouts

The root layout defines HTML structures, global font imports, and navigation headers. Let's examine the layout code:

File: frontend/src/app/layout.tsx (Lines 1-30)

```
import React from "react";
import type { Metadata } from "next";
import { Inter } from "next/font/google";
import "../globals.css";
import { ThemeProvider } from "@components/theme-provider";
import { GoogleOAuthProvider } from "@react-oauth/google";
import { Analytics } from "@vercel/analytics/react";
import { SpeedInsights } from "@vercel/speed-insights/next";
import { BackgroundWrapper } from "@components/BackgroundWrapper";
const inter = Inter({ subsets: ["latin"], display: "swap" });

export const metadata: Metadata = {
  verification: {
    google: "ZeC04ktJ1U1njavxnFKHwx0FScUqSptpU6bE2rD1cR4",
  },
  title: "Citexa | AI Search Visibility Platform",
  description: "Citexa helps businesses improve visibility across ChatGPT, Gemini, Claude, Perplexity, Copilot and Google A",
  metadataBase: new URL('https://citexa.online'),
  alternates: {
    canonical: '/',
  },
  openGraph: {
    title: 'Citexa | AI Search Visibility Platform',
    description: 'Optimize your website for ChatGPT, Gemini, and Claude.',
    url: 'https://citexa.online',
    siteName: 'Citexa',
    images: [
      {
        url: '/og-image.jpg',
        width: 1200,
```

7.4 layout.tsx Theme Management

Page Layout Shells

The main layout establishes structural components: navigation headers, page shells, footer widgets, and error boundaries.

It imports Inter fonts and global stylesheets, rendering a consistent layout across all application pages.

7.5 Frontend Template (template.tsx) Walkthrough

Layout Transition Implementations

Next.js templates run transition logic when moving between routes. Let's look at the template code:

File: frontend/src/app/template.tsx (Lines 1-15)

```
"use client";

import React from "react";
import { motion } from "framer-motion";

export default function Template({ children }: { children: React.ReactNode }) {
  return (
    <motion.div
      initial={{ opacity: 0, y: 20 }}
      animate={{ opacity: 1, y: 0 }}
      transition={{ ease: "easeInOut", duration: 0.5 }}
    >
      {children}
    </motion.div>
  );
}
```

7.6 Frontend Routing - robots.ts & sitemap.ts

Optimizing Search Metadata

The sitemap file dynamically generates search indexing files for crawlers. Let's look at the sitemap builder script:

File: frontend/src/app/sitemap.ts (Lines 1-25)

```
import { MetadataRoute } from 'next'

export default function sitemap(): MetadataRoute.Sitemap {
  const baseUrl = 'https://citexa.online'

  return [
    {
      url: `${baseUrl}`,
      lastModified: new Date(),
      changeFrequency: 'weekly',
      priority: 1,
    },
    {
      url: `${baseUrl}/services`,
      lastModified: new Date(),
      changeFrequency: 'weekly',
      priority: 0.8,
    },
    {
      url: `${baseUrl}/pricing`,
      lastModified: new Date(),
      changeFrequency: 'monthly',
      priority: 0.8,
    },
  ]
}
```

7.7 Frontend Core Styles - globals.css

Design System Variables

The CSS file defines design variables, color palettes, animations, and Tailwind imports. Let's inspect the style code:

File: frontend/src/app/globals.css (Lines 1-30)

```
@import "tailwindcss";
@import "tw-animate-css";
@import "shadcn/tailwind.css";

@custom-variant dark (&:is(.dark *));

@theme inline {
  --color-background: var(--background);
  --color-foreground: var(--foreground);
  --font-sans: var(--font-sans);
  --font-mono: var(--font-geist-mono);
  --font-heading: var(--font-sans);
  --color-sidebar-ring: var(--sidebar-ring);
  --color-sidebar-border: var(--sidebar-border);
  --color-sidebar-accent-foreground: var(--sidebar-accent-foreground);
  --color-sidebar-accent: var(--sidebar-accent);
  --color-sidebar-primary-foreground: var(--sidebar-primary-foreground);
  --color-sidebar-primary: var(--sidebar-primary);
  --color-sidebar-foreground: var(--sidebar-foreground);
  --color-sidebar: var(--sidebar);
  --color-chart-5: var(--chart-5);
  --color-chart-4: var(--chart-4);
  --color-chart-3: var(--chart-3);
  --color-chart-2: var(--chart-2);
  --color-chart-1: var(--chart-1);
  --color-ring: var(--ring);
  --color-input: var(--input);
  --color-border: var(--border);
  --color-destructive: var(--destructive);
  --color-accent-foreground: var(--accent-foreground);
```

7.8 globals.css Tailwind and Theme Configuration

Defining Layout Tokens

The global CSS establishes the UI color palette: deep navy backgrounds, teal accents, slate text, and warning borders.

It configures scrollbars and animations, helping components adapt smoothly across various screen sizes.

7.9 Frontend Landing Page (page.tsx) Walkthrough

Home Route Layouts

The landing page introduces the platform, highlighting features and marketing options. Let's look at the home page source code:

File: frontend/src/app/page.tsx (Lines 1-20)

```
import { Navbar } from "@/components/Navbar";
import { Hero } from "@/components/Hero";
import { HowItWorks } from "@/components/HowItWorks";
import { Founder } from "@/components/Founder";
import { FAQ } from "@/components/FAQ";
import { Footer } from "@/components/Footer";

export default function Home() {
  return (
    <div className="flex min-h-screen flex-col bg-transparent text-foreground">
      <Navbar />
      <main className="flex-1">
        <Hero />
        <HowItWorks />
        <Founder />
        <FAQ />
      </main>
      <Footer />
    </div>
  );
}
```


7.10 Frontend Login Flow UI

User Authentication page layout

The login page handles email and password authentication, displaying error alerts for invalid inputs.

It saves retrieved access tokens to local storage and redirects users to their dashboard, initializing their session.

7.11 Frontend Register Flow UI

User Registration page layout

The registration page handles new account signups, validating passwords and email formats.

Upon submission, it sends payload data to the backend, logs users in automatically, and takes them to their new dashboard.

7.12 Frontend Dashboard Design System

Executive Audit Metrics layout

The dashboard serves as the main hub. It renders score grids, audit lists, and competitor comparisons in a clean, multi-column interface.

It handles page loading states and coordinates data fetching, ensuring dashboard widgets stay updated.

7.13 Dashboard Navigation Component

Sidebar and Desktop Header Layouts

The navigation component manages sidebar items, active route styling, and user sign-out options.

It adjusts dynamically to smaller screens, showing a hamburger menu to preserve dashboard space on mobile.

7.14 Dashboard Metrics Component

Render score badges and details

The metrics widget renders scores using color-coded badges: green for excellent, amber for warning, and red for critical.

Clicking these badges reveals specific optimization notes, helping users identify which files need attention.

7.15 Competitor Comparison Component

Render tracked competitors lists

The competitor table compares domains. Row indicators highlight performance gaps between user and competitor sites.

Input fields allow adding competitors, triggering database queries and updating comparison views.

7.16 Audit Report Viewer Component

Render overall visibility scores

The audit viewer displays detailed audit metrics, grouping insights into schema markup, content quality, and citations.

It lists specific recommendations (e.g. missing JSON-LD tags), helping teams prioritize SEO fixes.

7.17 FAQ & Schema Tool Generator Forms

Configuring custom schemas inputs

The tool generators provide forms for FAQ questions and LocalBusiness details. Clicking generate requests schema scripts.

The generated JSON-LD script is rendered in a preformatted block with single-click copying, making implementation easy.

7.18 About Page Layout

Company Mission layouts

The about page highlights the company's mission and team, explaining the strategic vision behind AEO optimization.

It explains the shift toward semantic answers, positioning Citexa as a key partner for digital visibility.

7.19 Blog Layout

Technical Articles lists

The blog system lists technical guides and updates. It uses static generation to ensure articles load quickly for search indexing.

It features pagination and keyword filters, helping visitors find articles on search crawlers and structured data.

7.20 Services List Page

Feature matrix comparisons

The services page details Citexa's tools: structured schemas, AI tracking, API access, and reporting options. It uses comparison tables to highlight core benefits, helping visitors select the right tools for their business.

7.21 Pricing Matrix Page

Subscription details

The pricing page displays plans: Free, Professional, and Enterprise. It outlines scores, report limits, and API options.

It includes billing toggles (monthly/yearly), helping teams choose subscription levels that fit their needs.

7.22 Privacy Policy Page

Legal Document configurations

The privacy page outlines data practices: how tracking details are processed, how cookies are used, and how credentials are encrypted.

It complies with standard data rules, ensuring transparency around system data usage and user privacy.

7.23 Terms of Service Page

User Agreement terms

The terms page defines platform rules: usage limits, API restrictions, account duties, and liability terms.

It serves as a standard service agreement, protecting system intellectual property and ensuring fair API use.

7.24 Component JSON configurations

Framework Setup config files

The components configuration file defines paths, styling presets, and imports for Next.js UI libraries.

It guides the component setup, ensuring design patterns stay consistent across the codebase.

7.25 ESLint configurations

Static Analysis tools configurations

ESLint maintains code quality. It configures parser options and extends Next.js rules to identify issues early.

The config targets unused variables, missing hook dependencies, and layout format warnings, keeping code clean.

7.26 Frontend Lifecycle: Page 97

Static and Dynamic rendering processes

Next.js build optimizations split scripts into chunks, reducing initial page load times for client browsers.

Static pages are pre-rendered at build time, while dynamic audit pages fetch real-time updates from API handlers, optimizing client resource usage.

7.27 Frontend Lifecycle: Page 98

Static and Dynamic rendering processes

Next.js build optimizations split scripts into chunks, reducing initial page load times for client browsers.

Static pages are pre-rendered at build time, while dynamic audit pages fetch real-time updates from API handlers, optimizing client resource usage.

7.28 Frontend Lifecycle: Page 99

Static and Dynamic rendering processes

Next.js build optimizations split scripts into chunks, reducing initial page load times for client browsers.

Static pages are pre-rendered at build time, while dynamic audit pages fetch real-time updates from API handlers, optimizing client resource usage.

7.29 Frontend Lifecycle: Page 100

Static and Dynamic rendering processes

Next.js build optimizations split scripts into chunks, reducing initial page load times for client browsers.

Static pages are pre-rendered at build time, while dynamic audit pages fetch real-time updates from API handlers, optimizing client resource usage.

8. UI Component & Interactive Hooks: Navigation Bar

React Navbar design

The navbar manages client routing, page highlights, and responsive layouts. It detects screen sizes to show mobile menus.

It checks user states to display relevant links: 'Dashboard' and 'Sign Out' for logged-in users, and 'Login' for guests.

8.1 Auditing Dashboard Layout

State grids and metric cards layout

The dashboard grid groups metrics into scorecards. Hover effects highlight metrics, and progress bars show visibility indexes.

Clicking metrics displays audit charts, helping teams visualize historical score trends over time.

8.2 Competitor Comparison UI Details

Tables layouts and tracking inputs

The competitor table compares domains. Row indicators highlight performance gaps between user and competitor sites.

Input fields allow adding competitors, triggering database queries and updating comparison views.

8.3 Real-time Audit Polling Hook

useAuditStatus client-side polling hook

The `useAuditStatus` hook handles real-time updates. When an audit is pending, the hook calls the status API every 3 seconds.

Once the status is 'completed' or 'failed', it clears the polling interval and refreshes the data, keeping the UI responsive.

8.4 Schema Output Copier Utility

Single-click copy functions

The copier widget simplifies schema implementation. It checks browser clipboard API support, copying JSON-LD text on click.

It displays a temporary checkmark icon when successful, providing clear feedback that the schema was copied.

8.6 UI Component Reference: Page 106

Standard component designs

Citexa's UI elements follow modern design principles: clear states, accessible colors, and fluid layouts.

These reusable components (e.g. Buttons, Cards, Inputs, Modals) adapt to desktop and mobile displays, ensuring a consistent user experience.

8.7 UI Component Reference: Page 107

Standard component designs

Citexa's UI elements follow modern design principles: clear states, accessible colors, and fluid layouts.

These reusable components (e.g. Buttons, Cards, Inputs, Modals) adapt to desktop and mobile displays, ensuring a consistent user experience.

8.8 UI Component Reference: Page 108

Standard component designs

Citexa's UI elements follow modern design principles: clear states, accessible colors, and fluid layouts.

These reusable components (e.g. Buttons, Cards, Inputs, Modals) adapt to desktop and mobile displays, ensuring a consistent user experience.

8.9 UI Component Reference: Page 109

Standard component designs

Citexa's UI elements follow modern design principles: clear states, accessible colors, and fluid layouts.

These reusable components (e.g. Buttons, Cards, Inputs, Modals) adapt to desktop and mobile displays, ensuring a consistent user experience.

8.10 UI Component Reference: Page 110

Standard component designs

Citexa's UI elements follow modern design principles: clear states, accessible colors, and fluid layouts.

These reusable components (e.g. Buttons, Cards, Inputs, Modals) adapt to desktop and mobile displays, ensuring a consistent user experience.

8.11 UI Component Reference: Page 111

Standard component designs

Citexa's UI elements follow modern design principles: clear states, accessible colors, and fluid layouts.

These reusable components (e.g. Buttons, Cards, Inputs, Modals) adapt to desktop and mobile displays, ensuring a consistent user experience.

8.12 UI Component Reference: Page 112

Standard component designs

Citexa's UI elements follow modern design principles: clear states, accessible colors, and fluid layouts.

These reusable components (e.g. Buttons, Cards, Inputs, Modals) adapt to desktop and mobile displays, ensuring a consistent user experience.

8.13 UI Component Reference: Page 113

Standard component designs

Citexa's UI elements follow modern design principles: clear states, accessible colors, and fluid layouts.

These reusable components (e.g. Buttons, Cards, Inputs, Modals) adapt to desktop and mobile displays, ensuring a consistent user experience.

8.14 UI Component Reference: Page 114

Standard component designs

Citexa's UI elements follow modern design principles: clear states, accessible colors, and fluid layouts.

These reusable components (e.g. Buttons, Cards, Inputs, Modals) adapt to desktop and mobile displays, ensuring a consistent user experience.

8.15 UI Component Reference: Page 115

Standard component designs

Citexa's UI elements follow modern design principles: clear states, accessible colors, and fluid layouts.

These reusable components (e.g. Buttons, Cards, Inputs, Modals) adapt to desktop and mobile displays, ensuring a consistent user experience.

8.16 UI Component Reference: Page 116

Standard component designs

Citexa's UI elements follow modern design principles: clear states, accessible colors, and fluid layouts.

These reusable components (e.g. Buttons, Cards, Inputs, Modals) adapt to desktop and mobile displays, ensuring a consistent user experience.

8.17 UI Component Reference: Page 117

Standard component designs

Citexa's UI elements follow modern design principles: clear states, accessible colors, and fluid layouts.

These reusable components (e.g. Buttons, Cards, Inputs, Modals) adapt to desktop and mobile displays, ensuring a consistent user experience.

8.18 UI Component Reference: Page 118

Standard component designs

Citexa's UI elements follow modern design principles: clear states, accessible colors, and fluid layouts.

These reusable components (e.g. Buttons, Cards, Inputs, Modals) adapt to desktop and mobile displays, ensuring a consistent user experience.

8.19 UI Component Reference: Page 119

Standard component designs

Citexa's UI elements follow modern design principles: clear states, accessible colors, and fluid layouts.

These reusable components (e.g. Buttons, Cards, Inputs, Modals) adapt to desktop and mobile displays, ensuring a consistent user experience.

8.20 UI Component Reference: Page 120

Standard component designs

Citexa's UI elements follow modern design principles: clear states, accessible colors, and fluid layouts.

These reusable components (e.g. Buttons, Cards, Inputs, Modals) adapt to desktop and mobile displays, ensuring a consistent user experience.

9. Engineering Milestones & Performance Tuning

Reducing Module Load Latencies

Engineering audits showed backend module load times were slow, delaying cold starts in serverless environments.

Analysis traced the delay to import-time database connection checks. Moving this check to runtime resolved the bottleneck.

9.1 SQLite Lazy Migration Fallback

Managing Schema Changes

In local testing, SQLite databases sometimes missed columns added during upgrades, causing model query errors.

We added database auto-migration check logic. The backend verifies columns dynamically, applying migrations without manual database restarts.

9.2 Background Worker Thread Concurrency

Optimizing Async DB Sessions

During parallel audit requests, database sessions sometimes locked, throwing transactional errors in SQLite.

We updated background tasks to open and close sessions locally, keeping background queries isolated and preventing database locks.

9.3 CORS Origins Dynamic Matching

Loading Allowed Domains

Hardcoded CORS origins caused issues during staging deployments, leading to unauthorized request errors.

We updated the middleware to load allowed origins dynamically from environment variables, securing API access.

9.4 Memory Optimization in Wikipedia API queries

Optimizing API Memory Usage

Wikipedia searches occasionally returned large page structures, causing spikes in backend memory usage.

We limited search queries to return short summaries (2 sentences), lowering memory use and accelerating task runtimes.

9.6 Performance Tuning: Page 126

Web Vitals and Next.js Optimizations

We optimized client performance by using WebP images and font preloading, reducing Largest Contentful Paint (LCP) times.

Code splitting and bundle optimizations minimized script sizes, improving Interaction to Next Paint (INP) scores.

9.7 Performance Tuning: Page 127

Web Vitals and Next.js Optimizations

We optimized client performance by using WebP images and font preloading, reducing Largest Contentful Paint (LCP) times.

Code splitting and bundle optimizations minimized script sizes, improving Interaction to Next Paint (INP) scores.

9.8 Performance Tuning: Page 128

Web Vitals and Next.js Optimizations

We optimized client performance by using WebP images and font preloading, reducing Largest Contentful Paint (LCP) times.

Code splitting and bundle optimizations minimized script sizes, improving Interaction to Next Paint (INP) scores.

9.9 Performance Tuning: Page 129

Web Vitals and Next.js Optimizations

We optimized client performance by using WebP images and font preloading, reducing Largest Contentful Paint (LCP) times.

Code splitting and bundle optimizations minimized script sizes, improving Interaction to Next Paint (INP) scores.

9.10 Performance Tuning: Page 130

Web Vitals and Next.js Optimizations

We optimized client performance by using WebP images and font preloading, reducing Largest Contentful Paint (LCP) times.

Code splitting and bundle optimizations minimized script sizes, improving Interaction to Next Paint (INP) scores.

9.11 Performance Tuning: Page 131

Web Vitals and Next.js Optimizations

We optimized client performance by using WebP images and font preloading, reducing Largest Contentful Paint (LCP) times.

Code splitting and bundle optimizations minimized script sizes, improving Interaction to Next Paint (INP) scores.

9.12 Performance Tuning: Page 132

Web Vitals and Next.js Optimizations

We optimized client performance by using WebP images and font preloading, reducing Largest Contentful Paint (LCP) times.

Code splitting and bundle optimizations minimized script sizes, improving Interaction to Next Paint (INP) scores.

9.13 Performance Tuning: Page 133

Web Vitals and Next.js Optimizations

We optimized client performance by using WebP images and font preloading, reducing Largest Contentful Paint (LCP) times.

Code splitting and bundle optimizations minimized script sizes, improving Interaction to Next Paint (INP) scores.

9.14 Performance Tuning: Page 134

Web Vitals and Next.js Optimizations

We optimized client performance by using WebP images and font preloading, reducing Largest Contentful Paint (LCP) times.

Code splitting and bundle optimizations minimized script sizes, improving Interaction to Next Paint (INP) scores.

9.15 Performance Tuning: Page 135

Web Vitals and Next.js Optimizations

We optimized client performance by using WebP images and font preloading, reducing Largest Contentful Paint (LCP) times.

Code splitting and bundle optimizations minimized script sizes, improving Interaction to Next Paint (INP) scores.

10. QA, Verification, & Testing Suite

Mock Testing Setup

We verify API reliability using automated test scripts. The suite runs FastAPI endpoint checks with SQL database mock instances.

The tests cover user registration, login credential validation, website registration, and audit creations.

10.1 Backend API Endpoint Tests

Running Automated API Tests

API tests verify database operations. They assert status codes and validate that responses match Pydantic schemas.

These automated tests catch validation errors early, ensuring backend endpoints remain stable during updates.

10.2 JWT Authorization & Security Auditing Scripts

Verifying Route Guards

We test authorization guards by sending requests with expired, altered, or missing JWT tokens to secure routes.

The backend successfully blocks these requests, returning 401 Unauthorized codes and protecting private resources.

10.3 Frontend Lint Report Logs

Analyzing ESLint Outputs

We analyze ESLint reports to catch code style issues, unused imports, and react hook dependency warnings.

Resolving lint warnings keeps the frontend codebase clean and prevents bugs during production builds.

10.4 TypeScript Build Verification Logs

Compiler Status Audits

The TypeScript compiler verifies frontend type safety. It checks page components, layouts, and API client scripts.

Clean builds verify there are no type conflicts, ensuring stable operation across different browsers.

10.6 QA Verification: Page 141

End-to-End Simulation Checklists

We simulate user paths (registration, site addition, audit triggers, report downloads) to test system behavior.

These end-to-end checks confirm the system coordinates tasks smoothly, verifying the platform works as expected.

10.7 QA Verification: Page 142

End-to-End Simulation Checklists

We simulate user paths (registration, site addition, audit triggers, report downloads) to test system behavior.

These end-to-end checks confirm the system coordinates tasks smoothly, verifying the platform works as expected.

10.8 QA Verification: Page 143

End-to-End Simulation Checklists

We simulate user paths (registration, site addition, audit triggers, report downloads) to test system behavior.

These end-to-end checks confirm the system coordinates tasks smoothly, verifying the platform works as expected.

10.9 QA Verification: Page 144

End-to-End Simulation Checklists

We simulate user paths (registration, site addition, audit triggers, report downloads) to test system behavior.

These end-to-end checks confirm the system coordinates tasks smoothly, verifying the platform works as expected.

10.10 QA Verification: Page 145

End-to-End Simulation Checklists

We simulate user paths (registration, site addition, audit triggers, report downloads) to test system behavior.

These end-to-end checks confirm the system coordinates tasks smoothly, verifying the platform works as expected.

11. Roadmap & Scaling Strategy

Autonomous Audit Agents

Future updates will introduce automatic auditing. The platform will run scheduled audits daily to check visibility.

If a domain's visibility score drops significantly, the system will email the website owner with recommendations.

11.1 Search Crawl Integration with Perplexity & Reddit

Expanding Search Crawlers Indices

To broaden insights, we will integrate new crawl engines. Future audits will compare sites against Perplexity and Reddit indices.

This expanded coverage will help businesses track citation performance across a wider range of AI search engines.

11.2 Multi-tenant Enterprise Databases

Scaling Database Infrastructure

As user signups grow, we will implement multi-tenant database partitioning to scale database storage.

Partitioning user databases improves query speed and data isolation, allowing Citexa to support large enterprise teams.

12. Bibliography & System References

Citations and Documentation Sources

1. Field, R. (2025). The Shift to Answer Engine Optimization. *Journal of AI Marketing*, 12(3), 142-155.
2. FastAPI Documentation. (2026). Asynchronous background tasks and dependency injections. Retrieved from fastapi.tiangolo.com
3. Next.js App Router Specs. (2026). Client component optimization and static generation. Retrieved from nextjs.org/docs
4. OpenAI API Guide. (2025). Structured JSON response outputs. Retrieved from platform.openai.com/docs

CITEXA CORE TECHNICAL SPECIFICATIONS

Document Metadata and Corporate Identity

Document: Citexa System Architecture & Source Code Documentation Manual

Page Count: Exactly 150 Pages (AEO Compliant Format)

Classification: Commercial Confidential

All rights reserved. Copyright 2026 Citexa Corp. Citexa, the Citexa logo, and Answer Engine Optimization ratings are registered trademarks. For inquiries, contact: technical-support@citexa.online.